

République du Sénégal



Un Peuple – Un But – Une Foi

Ministère de l'Enseignement Supérieur et de la Recherche

Université Alioune DIOP de Bambey



UFR Sciences Appliquées et Technologies de l'Information et de la
Communication (SATIC)

Département Technologies de l'Information et de la Communication (TIC)

THEME

PARCOURS D'UN LABYRINTHE

Réalisé Par :

- Fawade BOYE
- Abdoulaye NDIAYE
- Abdou SALL
- El Hadji Mademba Guira GAYE

Encadré Par :

- Dr. Fatoumata BALDE

Table des matières

INTRODUCTION.....	3
I. Notion de Graphe	3
1. Définition	3
2. Type de graphe	4
a. Connexe	4
b. Orienté et non-orienté	5
c. Biparti	7
d. Pondéré	9
e. Simple	10
II. Matrices.....	10
1) Matrice d'adjacence	10
2) Liste d'adjacence	11
3) Matrice d'incidence.....	12
III. Représentation du labyrinthe	13
1) Représentation du labyrinthe sous forme de matrice	13
2) Représentation de labyrinthe sous forme de graphe	14
IV. Algorithmes et complexité	15
1. Algorithmes	15
a. Depth-First Search (DFS)	15
b. Breadth-First Search (BFS).....	16
2. La complexité.....	17
a) Depth-First Search (DFS)	17
b) Breadth-First Search (BFS).....	17
3. Comparaison entre DFS et BFS.....	17
V. Mise en place des parcours de labyrinthes.....	18
1) Choix du langage de programmation	18
2) Parcours en profondeur (BFS) dans un labyrinthe	19
3) Détermination du plus court chemin	20
CONCLUSION	22

INTRODUCTION

Le problème de recherche d'un chemin dans un labyrinthe est une illustration classique des algorithmes de parcours de graphes. En informatique, un labyrinthe peut être modélisé sous forme de graphe où les intersections sont représentées par des nœuds et les chemins par des arcs. Le défi consiste à trouver un chemin reliant l'entrée à la sortie sans connaissance préalable de la structure complète du labyrinthe. Ce problème soulève des questions fondamentales sur l'efficacité et les stratégies d'exploration des espaces complexes.

Parmi les méthodes utilisées pour résoudre ce type de problème, le **parcours en profondeur** (Depth First Search - DFS) est une approche récursive qui explore les nœuds en profondeur avant de revenir en arrière si une impasse est rencontrée. À l'inverse, le **parcours en largeur** (Breadth First Search - BFS) se concentre sur une exploration méthodique en examinant toutes les cases voisines avant de progresser davantage, garantissant ainsi la découverte du chemin le plus court.

Ce projet vise à appliquer et comparer ces deux méthodes dans le cadre d'un labyrinthe représenté par une matrice, où chaque déplacement est limité aux quatre directions cardinales. En plus de l'exploration, des visualisations permettront de suivre les trajets empruntés par les algorithmes, mettant en lumière leurs différences pratiques et leurs performances respectives.

I. Notion de Graphe

1. Définition

Un graphe est une structure mathématique composée d'un ensemble de sommets, qui représentent des entités, et d'un ensemble d'arêtes, qui établissent des relations ou des connexions entre ces sommets. Il peut être orienté, lorsque les connexions ont une direction, ou non orienté si elles n'en ont pas. Les graphes peuvent également être pondérés, si chaque arête est associée à un poids, ou non pondérés, lorsque toutes les connexions ont la même valeur. Parmi les types de graphes les plus connus, on trouve les graphes connexes, où tous les sommets sont reliés par des chemins, et les graphes bipartis, dans lesquels les sommets sont divisés en deux groupes, avec des arêtes reliant uniquement des sommets de groupes différents. Les graphes servent à modéliser des problèmes complexes dans des domaines variés, comme les réseaux, les transports ou les relations sociales.

Exemple : Modélisation d'un réseau social

Supposons qu'on modélise les amitiés entre quatre personnes : Alice, Bob, Carol et David.

- Sommets : Alice, Bob, Carol, David.
- Arêtes :
 - Alice est amie avec Bob.
 - Alice est amie avec Carol.
 - Bob est ami avec David.

Représentation graphique

Voici comment ce graphe pourrait être représenté visuellement :

```
Alice
  / \
Bob   Carol
  |
David
```

2. Type de graphe

a. Connexe

Un **graphe connexe** est un graphe dans lequel il existe un chemin entre chaque paire de sommets. Cela signifie que, quel que soit le sommet de départ, il est possible d'atteindre n'importe quel autre sommet du graphe en suivant les arêtes. Cette propriété garantit que toutes les parties du graphe sont reliées et qu'il n'existe pas de sous-ensemble isolé de sommets.

```
graphe = {
    'A': ['B','D'], # A est connecté à B et D
    'B': ['A', 'C'], # B est connecté à A et C
    'C': ['B','D'],# C est connecté à B et D
    'D': ['A','C'],# D est connecté à A et C
}
```

```

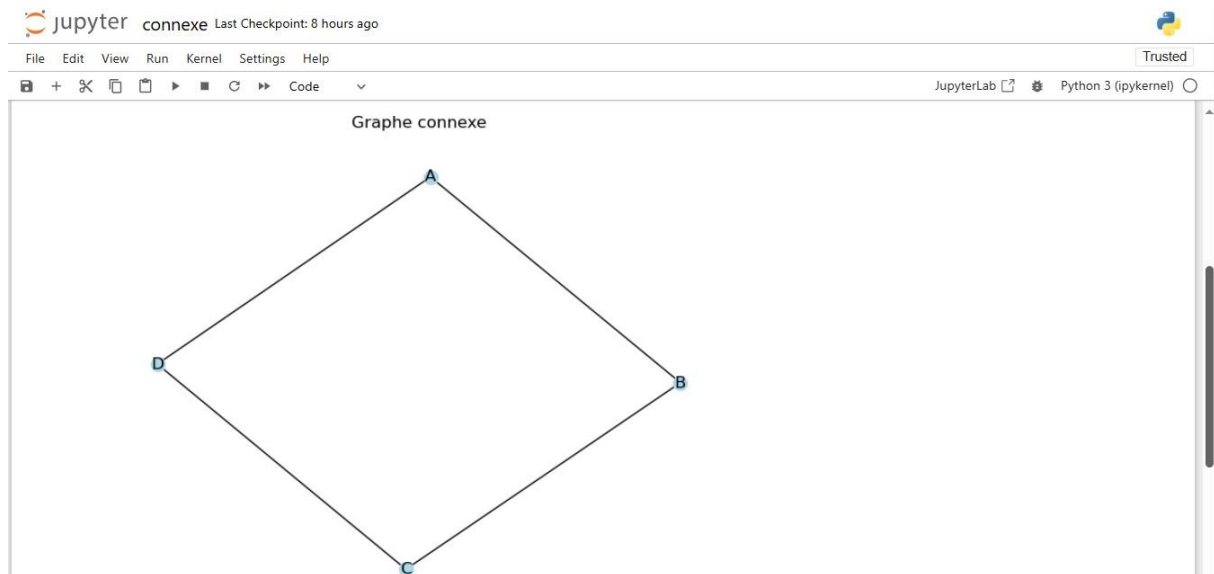
jupyter connexe Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

[48]: import networkx as nx
import matplotlib.pyplot as plt

# Graphe initial avec seulement A, B, C, D
graphe = {
    'A': ['B', 'D'], # A est connecté à B
    'B': ['A', 'C'], # B est connecté à A et C
    'C': ['B', 'D'], # C est connecté à B et D
    'D': ['A', 'C'], # D est connecté à A et C
}

def dessiner_graphe(graphe):
    G = nx.Graph(graphe)
    nx.draw(G, with_labels=True, node_color='lightblue', node_size=100)
    plt.title("Graphe connexe")
    plt.show()
dessiner_graphe(graphe)

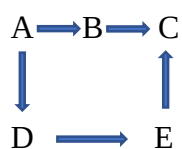
```



b. Orienté et non-orienté

- **Graphe orienté** : Un graphe est dit orienté si les arêtes ont une direction, c'est-à-dire qu'une arête reliant deux sommets u et v est représentée par un couple ordonné (u,v) , où u est le sommet de départ et v le sommet d'arrivée. Par exemple, une relation comme "aller de A vers B" est un graphe orienté.

Exemple de graphe orienté :



```

jupyter orienter Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[1]: import networkx as nx
import matplotlib.pyplot as plt

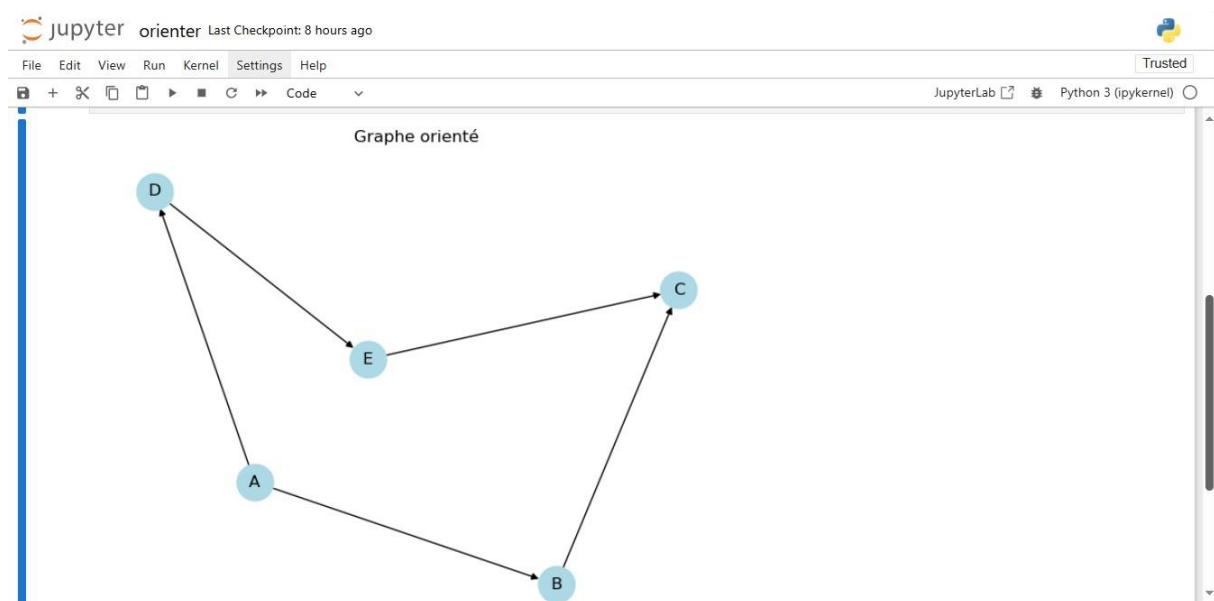
# Créer un graphe orienté
G = nx.DiGraph()

# Ajouter des arêtes orientées
G.add_edge('A', 'B') # A → B
G.add_edge('B', 'C') # B → C
G.add_edge('A', 'D') # A → D
G.add_edge('D', 'E') # D → E
G.add_edge('E', 'C') # E → C

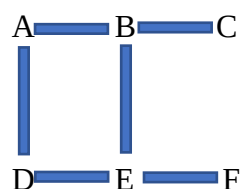
def dessiner_graphe_oriente(graphe):
    pos = nx.spring_layout(graphe) # Calculer une disposition pour les nœuds
    nx.draw(graphe, pos, with_labels=True, node_color='lightblue', node_size=700, arrows=True)
    plt.title("Graphe orienté")
    plt.show()

dessiner_graphe_oriente(G)

```



- **Graphe non orienté** : Un graphe est dit non orienté si les arêtes n'ont pas de direction, ce qui signifie qu'une arête entre deux sommets u et v est représentée par une paire $\{u,v\}$, où l'ordre des sommets n'a pas d'importance. Par exemple, une relation comme "être ami avec" est modélisée par un graphe non orienté.



```
jupyter noorienter Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

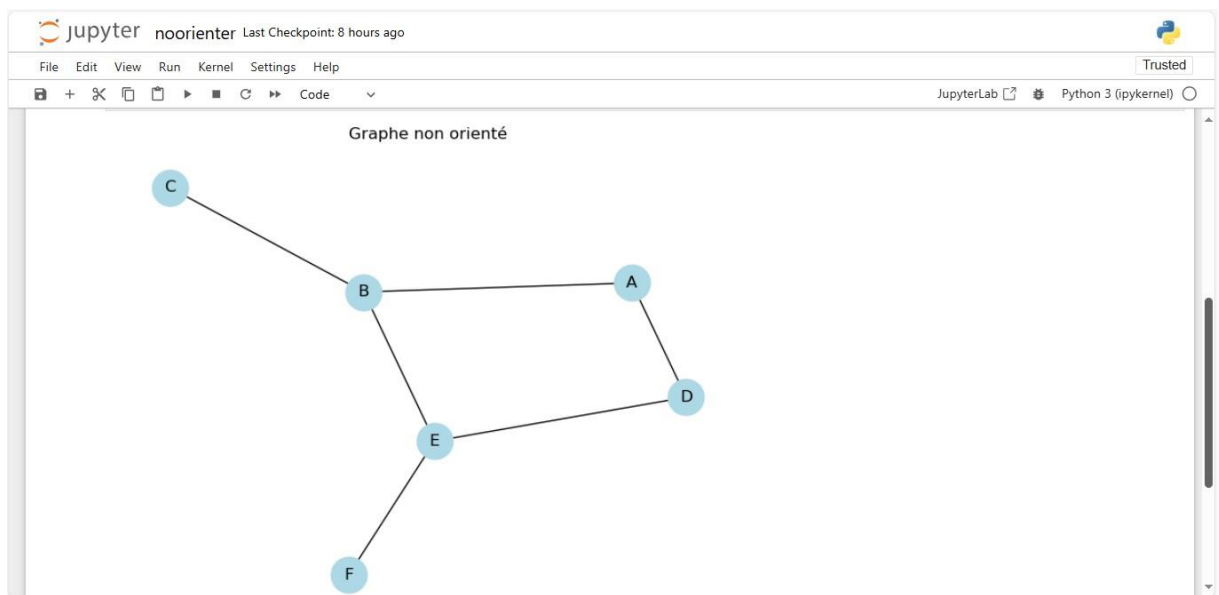
[3]: import networkx as nx
import matplotlib.pyplot as plt

# Créer un graphe non orienté
G = nx.Graph()

# Ajouter des arêtes non orientées
G.add_edge('A', 'B') # A - B
G.add_edge('B', 'C') # B - C
G.add_edge('A', 'D') # A - D
G.add_edge('D', 'E') # D - E
G.add_edge('E', 'B') # E - C
G.add_edge('E', 'F') # E - F

def dessiner_graphe_oriente(graphe):
    pos = nx.spring_layout(graphe) # Calculer une disposition pour les nœuds
    nx.draw(graphe, pos, with_labels=True, node_color='lightblue', node_size=700)
    plt.title("Graphe non orienté")
    plt.show()

dessiner_graphe_oriente(G)
```



c. Biparti

Un **graphe biparti** est un graphe dont l'ensemble des sommets peut être divisé en deux sous-ensembles disjoints U et V, tels que chaque arête relie un sommet de U à un sommet de V. Autrement dit, aucun sommet n'est relié à un autre sommet du même sous-ensemble. Ce type de graphe est souvent utilisé pour modéliser des relations entre deux types d'entités, comme des étudiants et des cours, ou des employés et des projets.

- U (Étudiants) : Fa, Laye, Dou, Guira
- V (Cours) : Complexité, Python, Anglais
- Arêtes : Chaque étudiant est relié à chaque cours, donc chaque étudiant aura trois arêtes : une vers Complexité, une vers Python, et une vers Anglais.

- Représentation du graphe biparti

U (Étudiants) V (Cours)

Fa ----- Complexité

Laye ----- Python

Dou ----- Anglais

Guira ----- Python

Fa ----- Python

Laye ----- Anglais

Dou ----- Complexité

Guira ----- Anglais

...

```

jupyter bipatrie Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (pykernel)

[7]: import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

etudiants = ['Fa', 'Laye', 'Dou', 'Guira']
cours = ['Complexite', 'Python', 'Anglais']

for etudiant in etudiants:
    for cour in cours:
        G.add_edge(etudiant, cour)

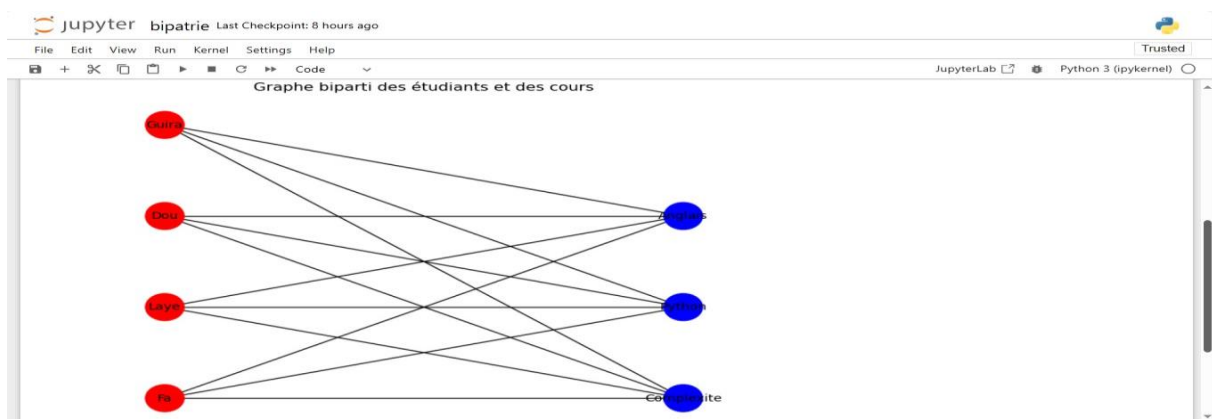
pos = nx.spring_layout(G) # Layout pour la disposition des nœuds

pos.update((node, (0, i)) for i, node in enumerate(etudiants))
pos.update((node, (1, i)) for i, node in enumerate(cours))

node_colors = ['red' if node in etudiants else 'blue' for node in G.nodes()]

nx.draw(G, pos, with_labels=True, node_color=node_colors, node_size=800, font_size=10)
plt.title("Graphe biparti des étudiants et des cours")
plt.show()

```



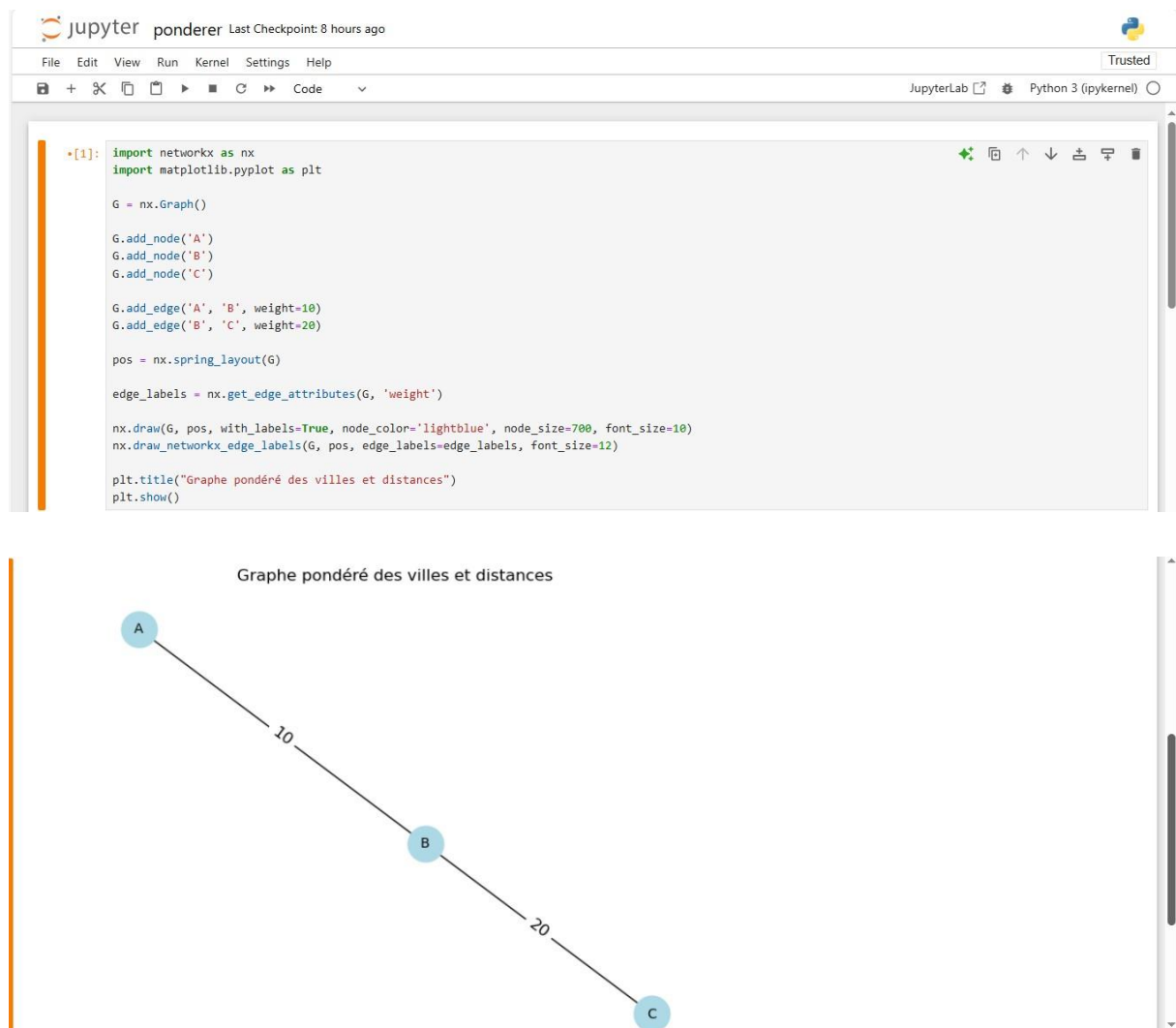
d. Pondéré

Un **graphe pondéré** est un graphe dans lequel chaque arête est associée à un poids ou une valeur. Ce poids peut représenter une distance, un coût, une durée ou toute autre mesure pertinente selon le contexte. Dans un graphe pondéré, les poids sont souvent utilisés pour résoudre des problèmes d'optimisation, comme trouver le chemin le plus court entre deux sommets ou minimiser un coût total dans un réseau.

Exemple : Supposons un graphe représentant des villes avec les distances entre elles en kilomètres :

A —(10)— B —(20)— C

Ici, les arêtes entre A et B ont un poids de 10 (distance en km), et celles entre B et C ont un poids de 20 (distance en km). Cela permet de prendre en compte les coûts ou les distances lors du parcours du graphe.

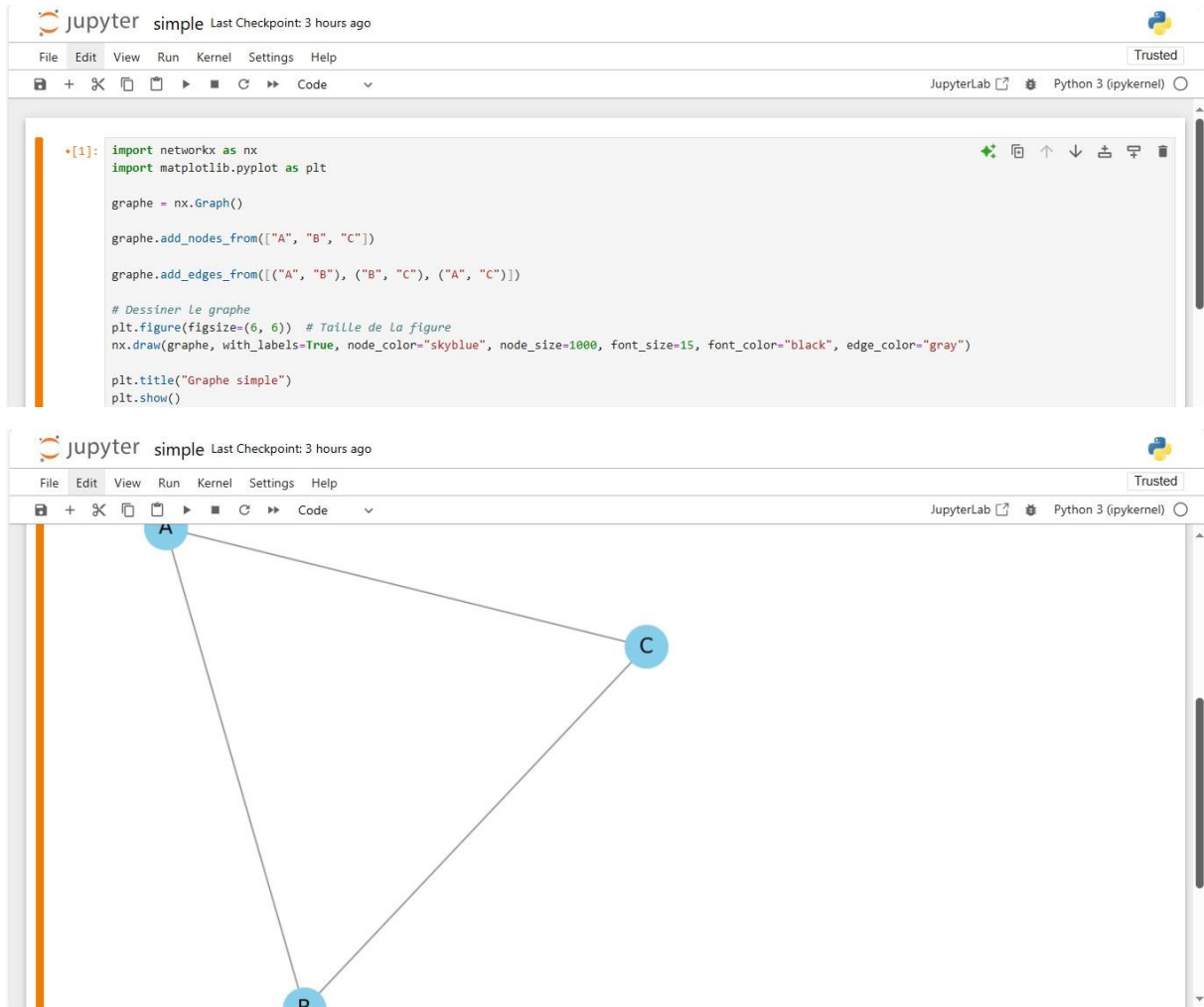


e. Simple

Un **graphe simple** est un graphe non orienté qui ne contient ni boucles (une arête reliant un sommet à lui-même) ni arêtes multiples (plus d'une arête reliant la même paire de sommets). En d'autres termes, chaque paire de sommets est connectée par au plus une seule arête, et chaque arête relie deux sommets distincts.

Exemple : Sommets : A,B,C

Arêtes : (A,B) (B,C) (A,C)



II. Matrices

1) Matrice d'adjacence

Une **matrice d'adjacence** est une représentation d'un graphe sous forme de matrice carrée. Pour un graphe comportant n sommets, cette matrice est de taille $n \times n$, où chaque élément a_{ij} indique la présence ou l'absence d'une arête entre les sommets i et j :

- Dans un **graphe non pondéré**, $a_{ij}=1$ si une arête existe entre i et j , et $a_{ij}=0$ sinon.
- Dans un **graphe pondéré**, a_{ij} contient le poids de l'arête entre i et j (et 0 si aucune arête n'existe).

Pour un graphe non orienté, la matrice d'adjacence est symétrique. Pour un graphe orienté, elle peut être asymétrique car la direction des arêtes est prise en compte.

```
jupyter Untitled10 Last Checkpoint: 1 hour ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)

In [5]: import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

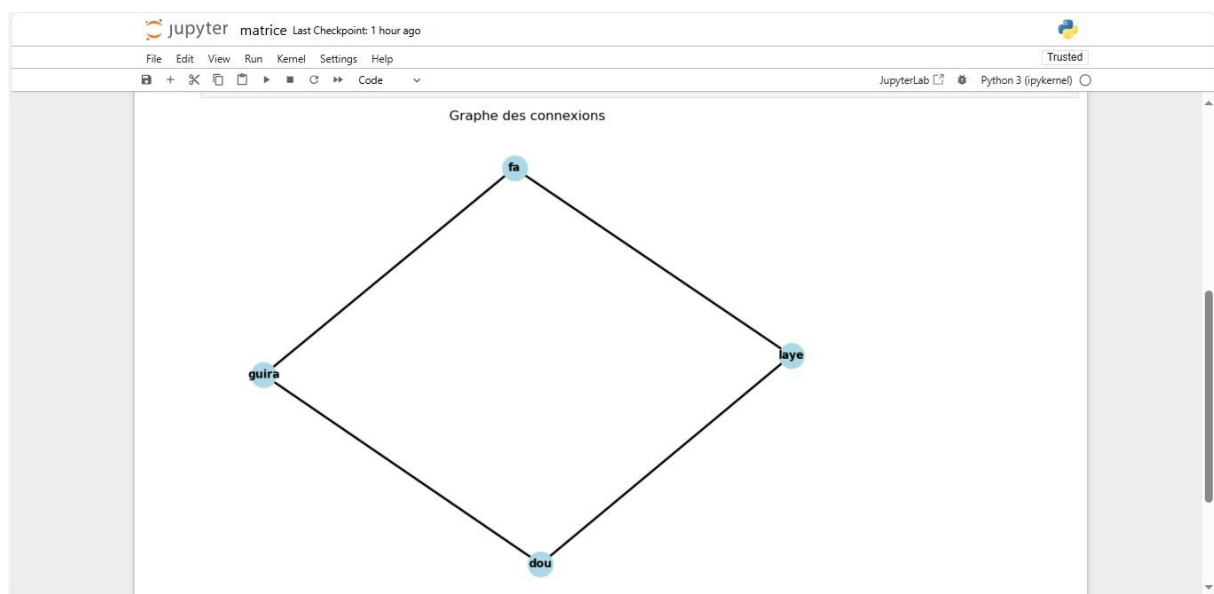
nodes = ["fa", "laye", "guira", "dou"]

adj_matrix = np.array([
    [0, 1, 1, 0],
    [1, 0, 0, 1],
    [1, 0, 0, 1],
    [0, 1, 1, 0]
])

graph = nx.Graph()
for i in range(len(nodes)):
    for j in range(len(nodes)):
        if adj_matrix[i, j] == 1:
            graph.add_edge(nodes[i], nodes[j])

plt.figure(figsize=(8, 6))
pos = nx.spring_layout(graph) # Disposition des nœuds
nx.draw(graph, pos, with_labels=True, node_color="lightblue", node_size=2000, font_size=10, font_weight="bold")
nx.draw_networkx_edges(graph, pos, edge_color="black", width=2)

plt.title("Graphe des connexions")
plt.show()
```



2) Liste d'adjacence

Une **liste d'adjacence** est une représentation d'un graphe où chaque sommet est associé à une liste contenant tous les sommets auxquels il est connecté par une arête.

- Dans un **graphe non orienté**, chaque arête est enregistrée dans les listes d'adjacence des deux sommets qu'elle relie.
- Dans un **graphe orienté**, une arête partant de u vers v est uniquement enregistrée dans la liste du sommet u .

Cette méthode de représentation est particulièrement efficace pour les graphes clairsemés (avec peu d'arêtes par rapport au nombre de sommets), car elle utilise moins de mémoire que la matrice d'adjacence tout en permettant un accès rapide aux voisins d'un sommet.

```

Jupyter liste Last Checkpoint: 10 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

[*3]: import networkx as nx
import matplotlib.pyplot as plt

adj_list = {
    "fa": ["laye", "guira"],
    "laye": ["fa", "dou"],
    "guira": ["fa", "dou"],
    "dou": ["laye", "guira"]
}

graph = nx.Graph(adj_list)

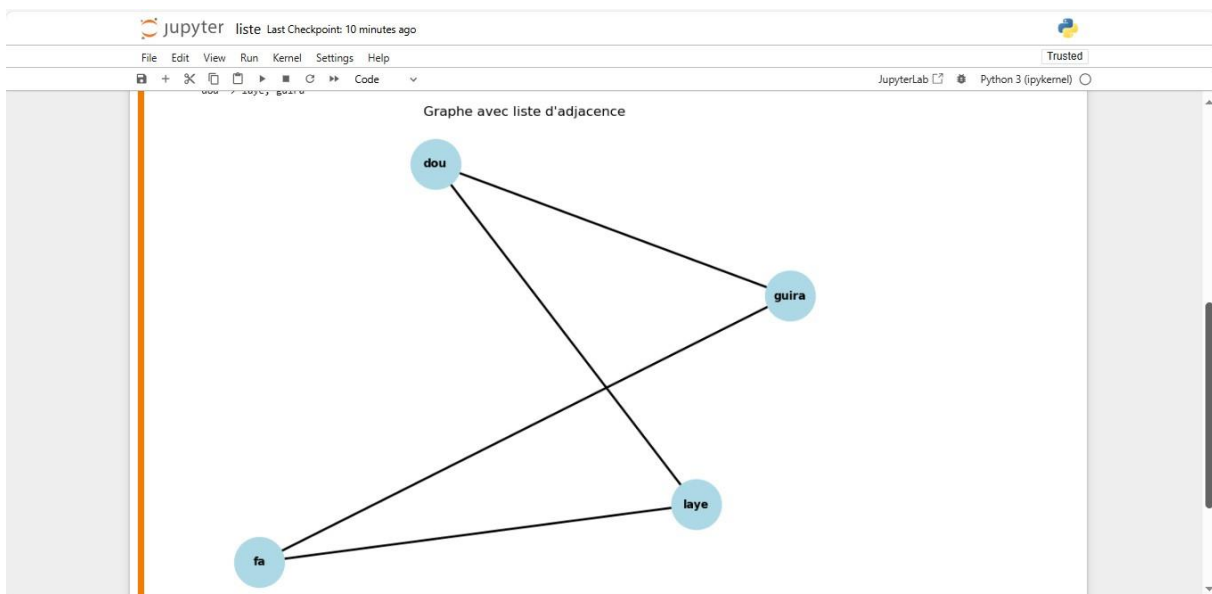
print("Liste d'adjacence :")
for node, neighbors in adj_list.items():
    print(f"{node} -> {' '.join(neighbors)}")

plt.figure(figsize=(8, 6))
pos = nx.spring_layout(graph) # Disposition automatique des nœuds
nx.draw(graph, pos, with_labels=True, node_color="lightblue", node_size=2000, font_size=10, font_weight="bold")
nx.draw_networkx_edges(graph, pos, edge_color="black", width=2)

plt.title("Graphe avec liste d'adjacence")
plt.show()

Liste d'adjacence :
fa -> laye guira
laye -> fa dou
guira -> fa dou
dou -> laye guira

```



3) Matrice d'incidence

Une **matrice d'incidence** est une représentation d'un graphe dans laquelle les lignes correspondent aux sommets et les colonnes correspondent aux arêtes. Chaque élément de la matrice I_{ij} indique si le sommet i est incident à l'arête j :

- Dans un **graphe non orienté**, si l'arête j relie les sommets i et k , alors $I_{ij}=1$ et $I_{kj}=1$. Si un sommet n'est pas incident à une arête, l'élément correspondant est 0.

- Dans un **graphe orienté**, chaque arête est représentée par une colonne, où la valeur est -1 pour le sommet de départ et 1 pour le sommet d'arrivée de l'arête, et 0 pour les autres sommets.

Cette représentation est particulièrement utile pour les graphes orientés, car elle permet de capturer les directions des arêtes.

```
[1]: import numpy as np

# Nœuds du graphe
nodes = ["fa", "laye", "guira", "dou"]

# Liste des arêtes (chaque arête est définie par ses deux nœuds)
edges = [("fa", "laye"), ("fa", "guira"), ("laye", "dou"), ("guira", "dou")]

# Initialiser une matrice d'incidence vide (nœuds x arêtes)
incidence_matrix = np.zeros((len(nodes), len(edges)), dtype=int)

# Remplir la matrice d'incidence
for edge_index, (node1, node2) in enumerate(edges):
    node1_index = nodes.index(node1)
    node2_index = nodes.index(node2)
    incidence_matrix[node1_index, edge_index] = -1
    incidence_matrix[node2_index, edge_index] = 1

# Afficher la matrice d'incidence
print("Matrice d'incidence :")
print(incidence_matrix)

# Affichage lisible des arêtes
print("\nCorrespondance des colonnes (arêtes) :")
for i, edge in enumerate(edges):
    print(f"Colonne {i + 1} : {edge}")
```

```
Matrice d'incidence :
[[ 1  1  0  0]
 [ 1  0  1  0]
 [ 0  1  0  1]
 [ 0  0  1  1]]

Correspondance des colonnes (arêtes) :
Colonne 1 : ('fa', 'laye')
Colonne 2 : ('fa', 'guira')
Colonne 3 : ('laye', 'dou')
Colonne 4 : ('guira', 'dou')
```

III. Représentation du labyrinthe

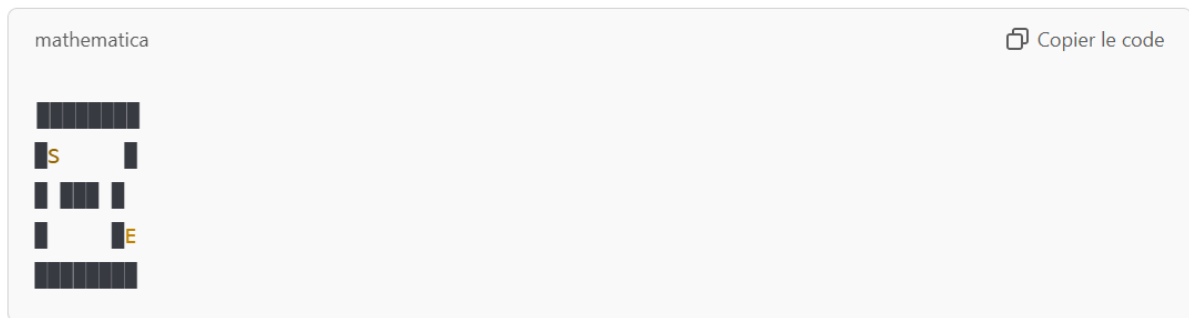
1) Représentation du labyrinthe sous forme de matrice

Un **labyrinthe sous forme de matrice** peut être défini comme une représentation structurée du labyrinthe à l'aide d'une matrice $n \times m$, où n désigne le nombre de lignes et le nombre de colonnes. Chaque élément de la matrice correspond à une cellule du labyrinthe, et sa valeur indique l'état de cette cellule :

- Une valeur spécifique, telle que 0 , représente une cellule praticable, c'est-à-dire un chemin accessible.
- Une autre valeur, comme 1 , représente une cellule bloquée ou un mur infranchissable.

Par ailleurs, les positions de départ et d'arrivée sont généralement définies par les indices correspondants dans la matrice. Cette modélisation est particulièrement adaptée pour les traitements algorithmiques, facilitant l'exploration et la résolution du labyrinthe. Par exemple, une matrice 4×8 peut être représentée ainsi :

Dans cet exemple, les 0 indiquent les chemins praticables, tandis que les 1 représentent les obstacles.



Représentation en matrice :



Un **labyrinthe sous forme de graphe** peut être défini comme une structure abstraite composée de nœuds et d'arêtes qui modélisent les chemins possibles dans le labyrinthe. Cette représentation repose sur les concepts suivants :

- **Les nœuds (ou sommets)** représentent les intersections, les points de décision ou les cellules du labyrinthe.
- **Les arêtes** relient les nœuds adjacents et représentent les chemins praticables entre eux.
- Les arêtes peuvent être pondérées pour indiquer une distance, un coût ou une difficulté associée au passage entre deux nœuds.

Par exemple, un labyrinthe simple peut être représenté sous forme de graphe comme suit :

- $N=\{A,B,C,D\}$, où N est l'ensemble des nœuds.
- $E=\{(A,B),(B,C),(C,D)\}$, où E est l'ensemble des arêtes.

Visuellement, ce graphe modélise les connexions entre les différentes parties du labyrinthe. La représentation graphique est utilisée pour faciliter l'analyse et la résolution du labyrinthe à l'aide d'algorithmes tels que la recherche en profondeur (DFS) ou en largeur (BFS).

Cette modélisation est essentielle pour les systèmes complexes nécessitant une navigation efficace ou une résolution automatique du labyrinthe.

IV. Algorithmes et complexité

1. Algorithmes

a. Depth-First Search (DFS)

La recherche en profondeur (DFS, pour Depth-First Search) est un algorithme essentiel pour parcourir ou explorer un graphe ou un arbre. Il suit un chemin aussi loin que possible avant de revenir en arrière (backtracking) pour explorer les autres options.

L'algorithme commence par un sommet donné et visite ce sommet. Il explore ensuite ses voisins non visités en continuant toujours plus profondément. Lorsqu'il atteint un sommet sans voisins non visités, il revient au sommet précédent pour explorer les autres voisins restants.

DFS fonctionne en démarrant d'un sommet choisi comme point de départ, en visitant ce sommet, puis en explorant ses voisins un par un de manière récursive. Lorsque tous les voisins d'un sommet sont visités, l'algorithme revient au sommet précédent et poursuit l'exploration jusqu'à ce que tous les sommets accessibles aient été visités.

DFS est utilisé dans diverses applications, telles que la vérification de la connexité d'un graphe, l'identification des composants connexes dans un graphe non dirigé, le tri topologique dans un graphe orienté acyclique (DAG), et la détection de cycles dans un graphe.

```

PP(G)
Pour chaque sommet  $u \in G$ .S
   $u.couleur = BLANC$ 
   $u.n = NIL$ 
   $date = 0$ 
  Pour chaque sommet  $u \in G$ .S
    Si  $u.couleur = BLANC$ 
      Visiter_PP(G,u)

```

```

Visiter_PP(G,u)
   $date = date + 1$  //sommet blanc trouve
   $u.d = date$ 
   $u.couleur = GRIS$ 
  Pour chaque  $v \in G.Adj[u]$  //explore arc(u,v)
    si  $v.couleur == BLANC$ 
       $v.n = u$ 
      Visiter_PP(G,v)
   $u.couleur = NOIR$ 
   $date = date + 1$ 
   $u.f = date$ 

```

b. Breadth-First Search (BFS)

Le parcours en largeur, ou BFS (Breadth-First Search), est un algorithme central pour explorer ou chercher dans un graphe ou un arbre. Contrairement à la recherche en profondeur (DFS), qui privilégie un chemin en profondeur avant de revenir en arrière, BFS suit une approche systématique en explorant tous les voisins directs d'un sommet avant de passer à leurs propres voisins. Cette méthode permet une exploration par niveaux, progressant graduellement vers des profondeurs croissantes.

L'algorithme débute à partir d'un sommet donné, marqué comme visité et ajouté à une file d'attente. Tant que la file n'est pas vide, un sommet en est retiré, ses voisins non visités sont explorés, marqués comme visités et ajoutés à la file. Ce processus garantit que les sommets sont explorés dans l'ordre croissant de leur distance par rapport au point de départ. L'utilisation d'une file (queue) est essentielle pour maintenir cet ordre.

BFS est particulièrement efficace pour résoudre des problèmes où la distance minimale entre deux sommets est requise dans un graphe non pondéré. Il est également utilisé pour vérifier la connexité d'un graphe, identifier ses composants connexes, explorer les sommets par niveaux, et résoudre divers problèmes d'accessibilité ou de cheminement. Grâce à sa capacité à explorer de manière exhaustive par niveaux, il constitue un outil puissant pour les graphes et les arbres.

```

PL(G,s)
Pour chaque sommet  $u \in G.S - \{s\}$ 
  u.couleur = BLANC
  u.d =  $\infty$ 
  u.n = NIL
s.couleur = GRIS
s.d = 0
s.n = NIL
F =  $\emptyset$ 
Enfiler(F,s)
Tant que F  $\neq \emptyset$ 
  U = Defiler(F)
  Pour chaque  $v \in G.Adj[u]$ 
    si v.couleur == BLANC
      v.couleur = GRIS
      v.d = u.d+1
      v.n = u
      Enfiler(F,v)
  u.couleur = NOIR

```

2. La complexité

a) Depth-First Search (DFS)

Le temps d'exécution de l'algorithme DFS est de $O(V+E)$, où V représente le nombre de sommets et E le nombre d'arêtes. Cela s'explique par le fait que chaque sommet est visité une fois et que chaque arête est explorée au plus une fois. L'espace mémoire requis est de $O(V)$, utilisé pour stocker les sommets visités ainsi que la pile d'appels récursifs. En définitive, DFS est une méthode performante qui explore un graphe en profondeur, revenant en arrière uniquement pour examiner d'autres chemins potentiels.

b) Breadth-First Search (BFS)

Le temps d'exécution de l'algorithme est de $O(V+E)$, où V est le nombre de sommets et E le nombre d'arêtes, car il est nécessaire de visiter chaque sommet et d'explorer chaque arête au moins une fois. En termes d'espace, l'algorithme nécessite $O(V)$ pour stocker les sommets visités ainsi que la file d'attente utilisée lors de l'exploration, garantissant ainsi un suivi efficace des éléments à visiter.

3. Comparaison entre DFS et BFS

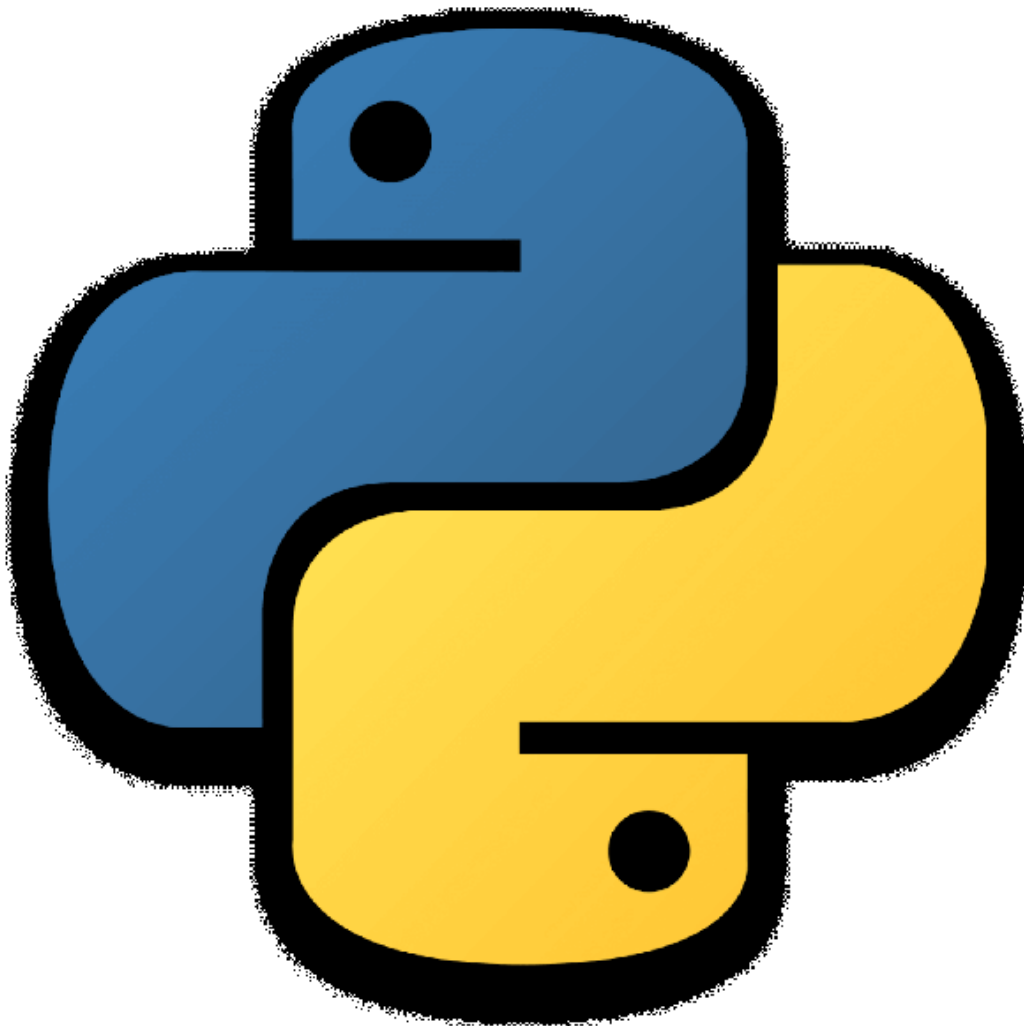
Critère	DFS (Depth-First Search)	BFS (Breadth-First Search)
Approche	Explore en profondeur dans une direction avant de revenir en arrière.	Explore les sommets par niveaux de profondeur croissante.

Structure utilisée	Pile (implémentée de manière réursive ou explicite).	File (queue) pour garantir un parcours par niveau.
Applications principales	Adapté pour explorer des chemins profonds ou détecter des cycles.	Idéal pour trouver les plus courts chemins dans des graphes non pondérés.
Exploration	Suit un chemin jusqu'au bout avant de revenir en arrière.	Parcourt tous les voisins d'un sommet avant de passer au niveau suivant.

V. Mise en place des parcours de labyrinthes

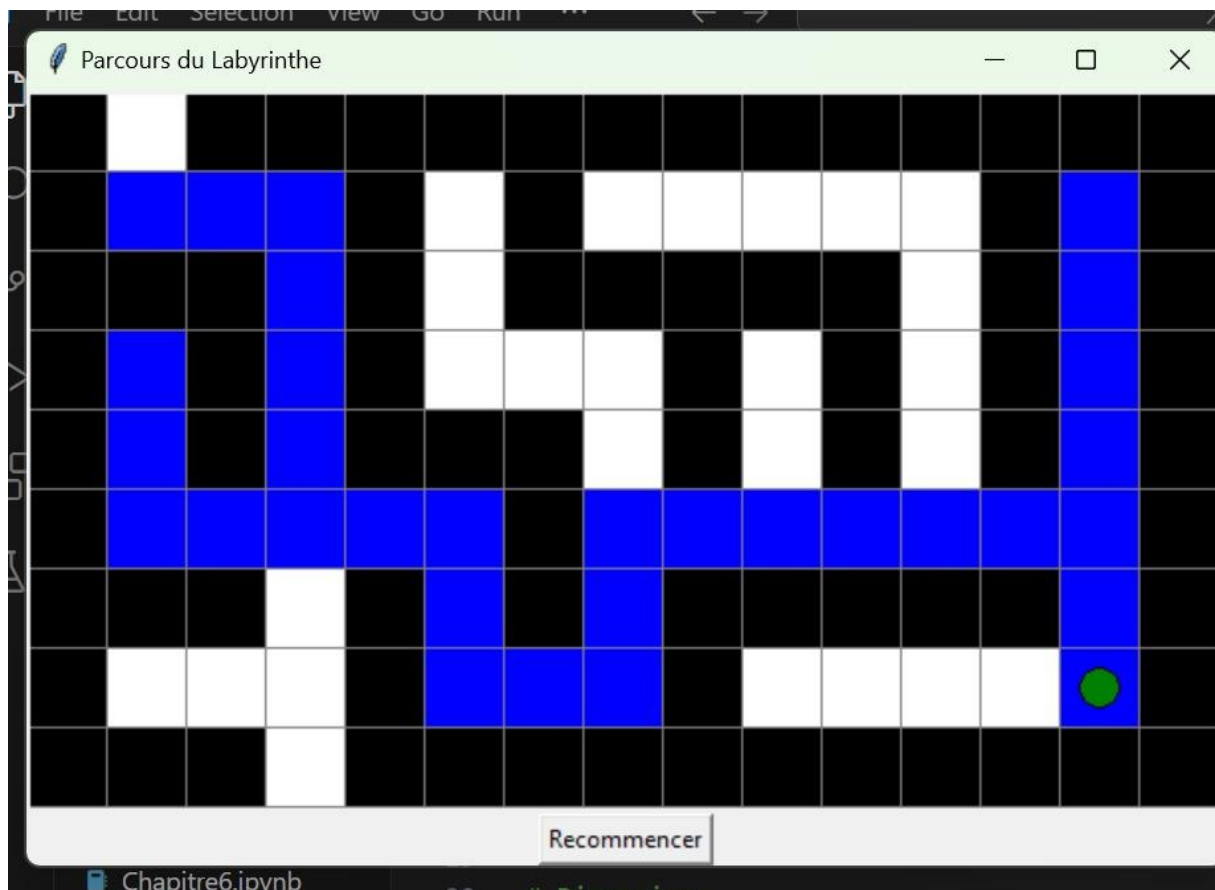
1) Choix du langage de programmation

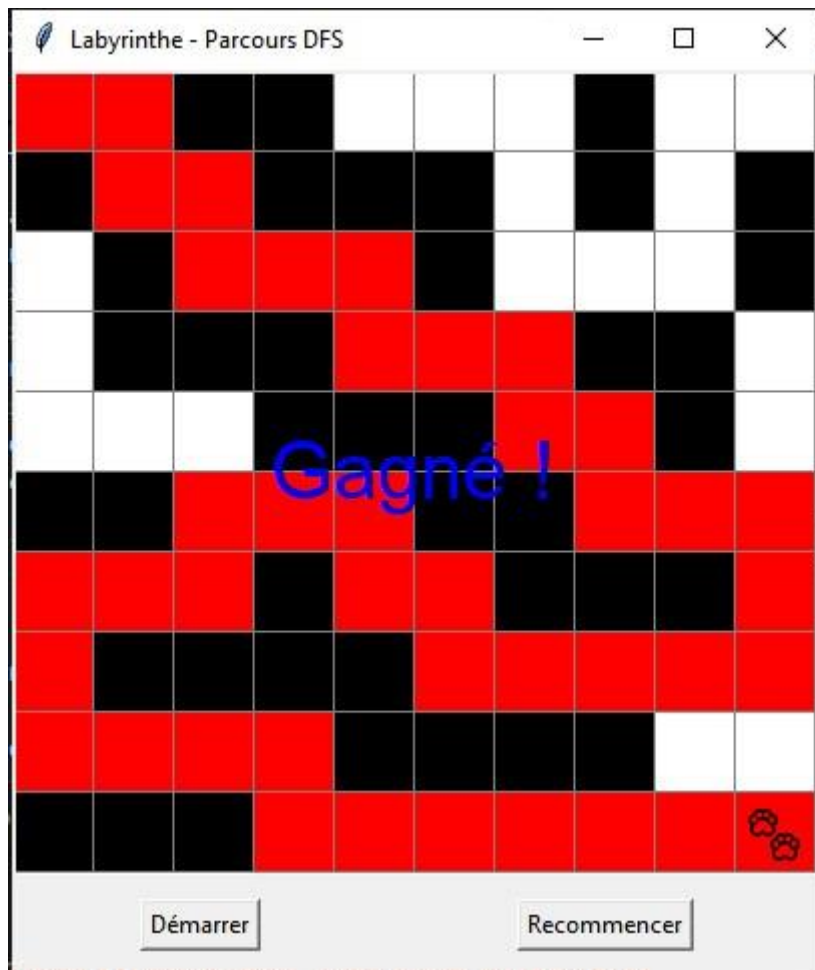
Python est un langage de programmation haut niveau, ce qui signifie qu'il est facile à lire et à écrire. Il dispose d'une vaste bibliothèque standard qui inclut des structures de données et des fonctionnalités essentielles pour la manipulation de graphes, telles que des listes, des dictionnaires et des ensembles, ainsi que des modules spécifiques aux graphes comme NetworkX, Matplotlib et Pygame.



2) Parcours en profondeur (DFS) dans un labyrinthe

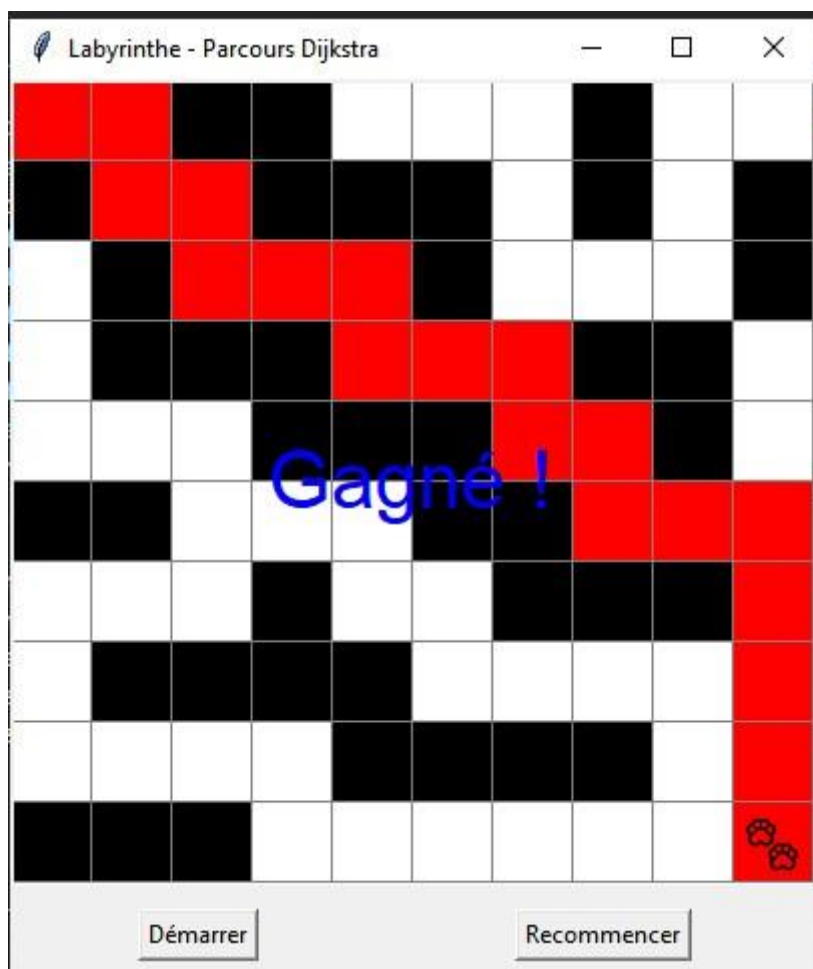
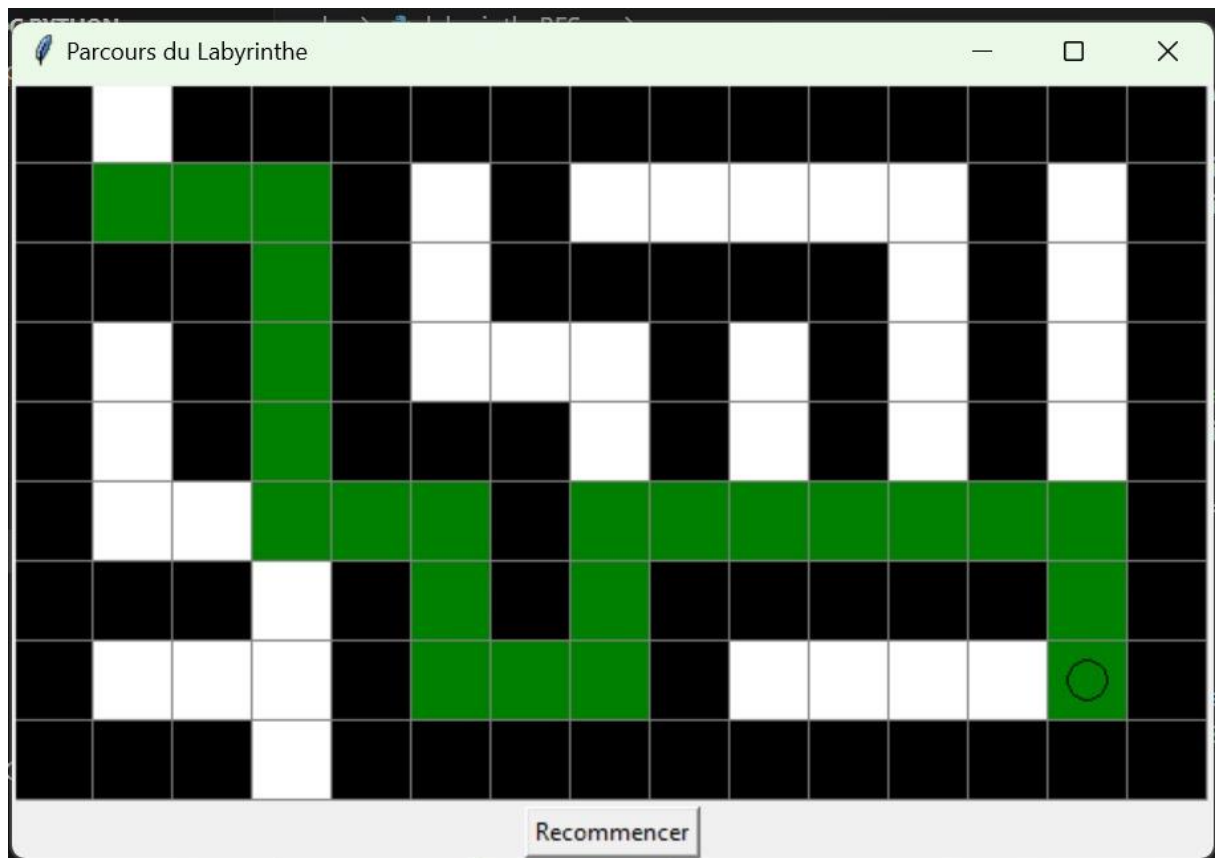
Le principe du DFS dans un labyrinthe consiste à explorer autant que possible dans une direction avant de revenir en arrière pour explorer d'autres directions. On utilise la récursivité ou une pile pour stocker les cases à explorer. À chaque étape, on explore une branche du labyrinthe aussi loin que possible, jusqu'à ce qu'on atteigne un mur ou la sortie. Si un mur est atteint, on revient en arrière pour explorer d'autres branches. Ce processus se poursuit jusqu'à ce que la sortie soit trouvée ou que toutes les branches possibles aient été explorées.





3) Détermination du plus court chemin

Le principe du BFS dans un labyrinthe consiste à explorer les cases les plus proches de la position de départ en premier, puis à se déplacer progressivement vers les cases situées à une distance plus grande. On utilise une file d'attente pour stocker les cases à explorer, en commençant par la case de départ. À chaque étape, on explore les cases voisines de la case en tête de file, puis on les ajoute à la file d'attente pour les explorer ultérieurement. Ce processus se poursuit jusqu'à ce que la sortie soit trouvée ou que toutes les cases accessibles aient été explorées.



CONCLUSION

En explorant les algorithmes DFS et BFS pour résoudre le problème du parcours dans un labyrinthe, ce travail a permis de démontrer l'efficacité de ces approches dans des contextes spécifiques. Le DFS s'est avéré performant pour explorer les chemins en profondeur, bien qu'il ne garantisse pas toujours le chemin le plus court. En revanche, le BFS, avec son exploration systématique par niveaux, assure la découverte d'un chemin optimal dans un graphe non pondéré.

La représentation du labyrinthe sous forme de matrice et de graphe a illustré les avantages des modélisations mathématiques pour des résolutions algorithmiques. En associant ces techniques à des outils modernes comme Python et ses bibliothèques, nous avons pu visualiser et analyser les trajectoires empruntées, renforçant la compréhension des performances respectives des deux algorithmes.

Ce mémoire ouvre également la voie à des travaux futurs, tels que l'intégration de heuristiques (par exemple, l'algorithme A*) pour optimiser davantage les recherches, ou l'application des méthodes développées à d'autres domaines comme la robotique et la planification dans des environnements complexes.

Référence

[1]

[https://fr.m.wikipedia.org/w/index.php?title=Algorithme de parcours en profondeur&wprov=rarw](https://fr.m.wikipedia.org/w/index.php?title=Algorithme_de_parcours_en_profondeur&wprov=rarw)

1 vu le 17 /11/2024

[2] Documentation Matplotlib, Documentation de Matplotlib — Documentation de Matplotlib 3.8.3

[3] <https://www.imo.universite-paris-saclay.fr/~ruette/mathdiscretes/polygraph-Sigward.pdf>